

Energy Efficiency

Jimit Gosar, Ishaan Mathur, Anay Bhakat, Louis Asanaka,
Georges Durand

Perseus: Reducing Energy Bloat in Large Model Training

- Training large AI models consumes a massive amount of energy
- Not all energy consumed contributes to end-to-end throughput
- Amazon's training of a 200B model, consumed about 11.9 GWh 1,000 average US households for a year

Goal: Reduce energy bloat without slowing down training

Intrinsic energy bloat

Computation imbalance when a large model is distributed across multiple GPUs with pipeline parallelism

Perfectly balancing every stage is not always possible

Non-critical computations run unnecessarily fast, wasting energy

Extrinsic energy bloat

Occurs when straggler pipelines delay synchronization.

Non-straggler pipelines waste energy by running faster than necessary.



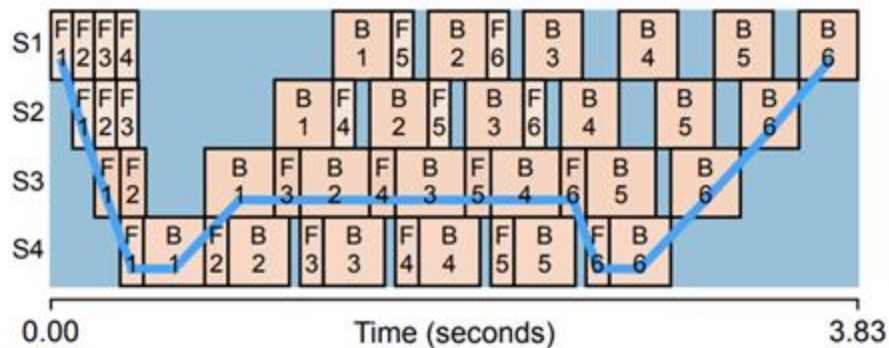
LLM Training techniques

3D Parallelism:

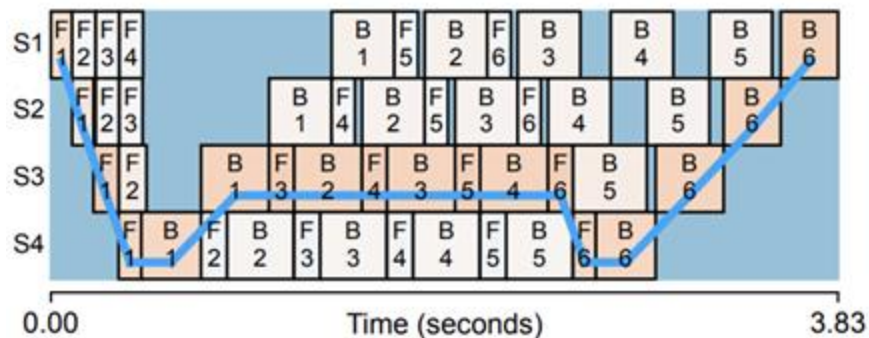
- Data Parallelism: Split dataset across devices
- Tensor Parallelism: Split model tensors across devices
- Pipeline Parallelism: Split model layers into stages, process microbatches

Focus: Pipeline parallelism is key for large models

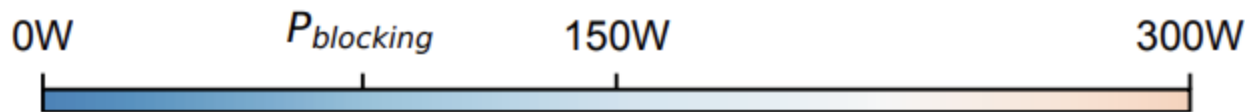
Example with GPT-3 1.3B



(a) Execution timeline of one training iteration



(b) Execution timeline with reduced intrinsic energy bloat



Imbalance in Pipeline Stages

Reason: Coarse-grained tensor operations

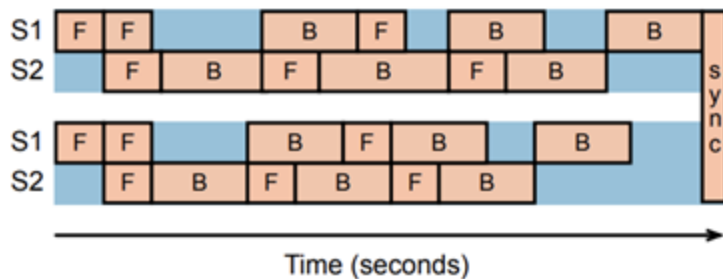
Perfect balance is NP-hard (even to approximate) due to layer granularity

Trend: Imbalance increases with more stages

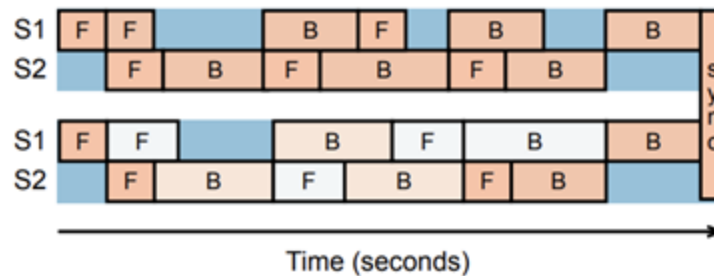
Model	# Parameters	Imbalance Ratio	
		4 stages	8 stages
GPT-3 [8]	3B	1.13	1.25
	7B	1.11	1.23
	13B	1.08	1.17
	175B	1.02	1.03
Bloom [82]	3B	1.13	1.25
	7B	1.13	1.25
	176B	1.05	1.10
BERT [19]	0.1B	1.33	2.00
	0.3B	1.17	1.33
T5 [66]	0.2B	1.19	1.50
	0.7B	1.05	1.11
	2.9B	1.06	1.16
Wide-ResNet50 [87]	0.8B	1.23	1.46
Wide-ResNet101 [87]	1.5B	1.09	1.25

Table 1. Forward latency ratio of the longest to the shortest stage on A100 GPUs. 1.00 would mean perfect balance.

Stragglers in Training Pipelines



(a) Extrinsic energy bloat caused by a straggler



(b) Reduced intrinsic and extrinsic energy bloat

Impact: Non-straggler pipeline runs full speed, wastes energy

Solution: Slow non-stragglers to match straggler time

Likelihood: Increases with scale (more GPUs, longer jobs)

Perseus

Upper Bound: Slow all computations to minimum-energy frequencies

A100 GPUs: 16% energy reduction

A40 GPUs: 27% energy reduction

Approach:

Characterizes time-energy tradeoff frontier

Schedules computations to optimize energy use

Unified Optimization Framework

Objective: Minimize energy while meeting iteration time constraints

Formulation:

Minimize Energy(F) subject to $\text{Time}(F) \leq T'$

F = frequency assignment for computations

Core Idea: Use time-energy frontier to balance intrinsic and extrinsic bloat

Expression for Energy Consumption

$$= \sum_i (e_i(f_i) - P_{\text{blocking}} \cdot t_i(f_i)) + P_{\text{blocking}} \cdot N \cdot T'$$

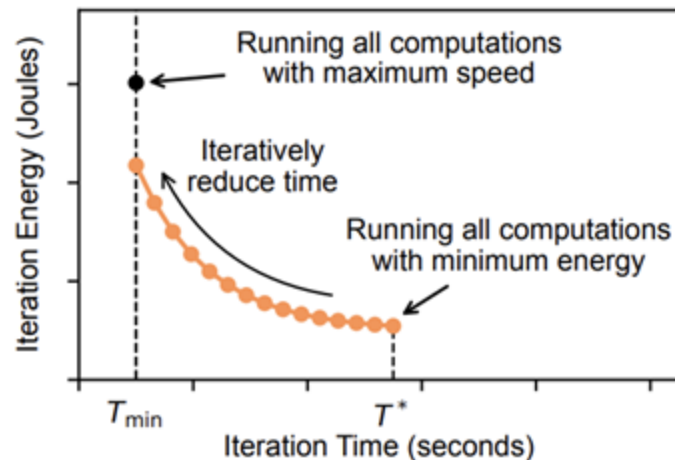
Finding the minimum energy iteratively

$\min \text{Energy}(F) \text{ s.t. } \text{Time}(F) \leq T'$

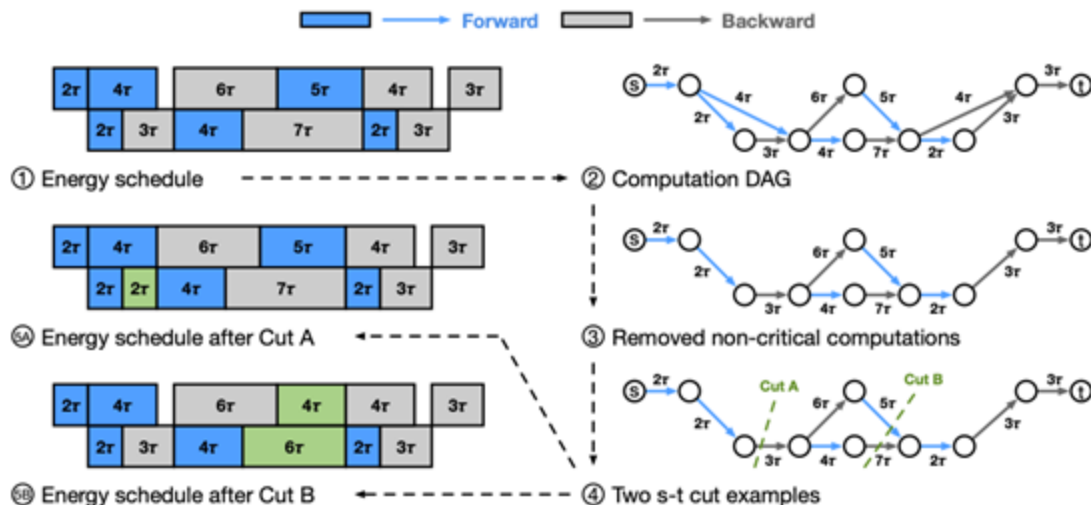
NP-hard is because it is an optimization problem where the possible choices of computation time and energy are discrete

GPUs only support discrete frequencies

```
► Begin with the minimum energy schedule
1  $s \leftarrow$  Minimum energy for all computations
2  $S \leftarrow \{s\}$ 
3 while  $\text{IterationTime}(\mathcal{G}, s) > T_{\min}$  do
    ► Reduce time by  $\tau$  with minimal energy increase
4      $s \leftarrow \text{GetNextSchedule}(\mathcal{G}, s, \tau)$ 
5      $S \leftarrow S \cup \{s\}$ 
6 return  $S$ 
```



Minimum Cut



Input: DAG $\mathcal{G}$ of computations $i \in \mathcal{G}$

Current energy schedule s

Amount of iteration time to reduce τ

Output: Neighbor schedule with reduced time s'

- Construct edge-centric computation DAG (②)
- 1 $\mathcal{G} \leftarrow$ Split nodes into two and connect with edge
- Find and remove non-critical computations (③)
- 2 Annotate earliest & latest start times for $\forall i \in \mathcal{G}$
- 3 **for** $i \in \mathcal{G}$ **do**
- 4 **if** i has different earliest and latest start times **then**
- 5 Remove i from $\mathcal{G}$
- Find set of computations to modify (④)
- 6 $S, T \leftarrow \text{FindMinCut}(\mathcal{G}, s)$
- Modify computation durations (⑤)
- 7 Modify duration of $\forall i$ in $S - T$ cut by τ
- Assign frequencies from planned computation times
- 8 $s' \leftarrow \min f_i$ that runs no slower than planned
- 9 **return** s'

Algorithm 2: GetNextSchedule: Reducing the execution time of the DAG by τ with minimal energy increase.

Time energy tradeoff frontier

Model	Energy Savings (%)		Slowdown (%)	
	Perseus	EnvPipe	Perseus	EnvPipe
GPT-3 1.3B	13.2	8.8	0.1	0.1
BERT 1.3B	12.9	8.0	0.5	0.0
T5 3B	10.6	7.4	1.3	3.4
Bloom 3B	11.7	8.9	0.2	0.2
Wide-ResNet 1.5B	3.2	3.7	2.3	4.1

(a) Four stage pipeline parallelism on A100 GPUs

Model	Energy Savings (%)		Slowdown (%)	
	Perseus	EnvPipe	Perseus	EnvPipe
GPT-3 2.7B	21.1	21.7	0.2	5.6
BERT 1.3B	15.7	16.5	0.0	9.7
T5 3B	28.5	19.3	0.0	0.0
Bloom 3B	22.4	19.9	0.0	0.0
Wide-ResNet 1.5B	20.4	16.5	0.2	0.5

(b) Eight stage pipeline parallelism on A40 GPUs

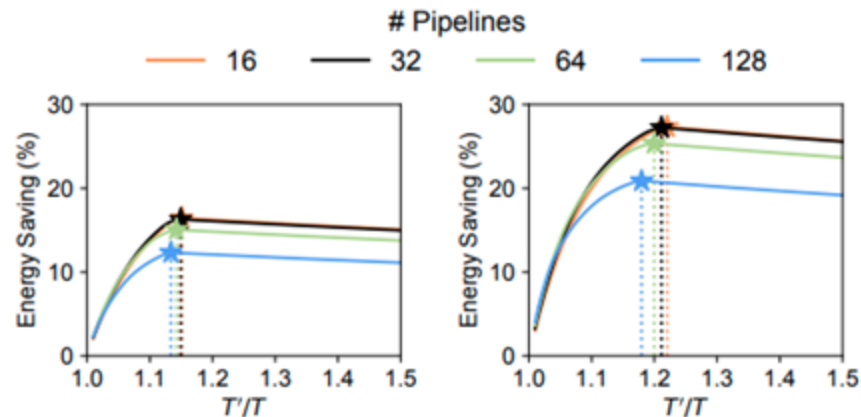
Table 3. [Experiment] Intrinsic energy bloat (without stragglers) reduction and iteration time slowdown.

Model # Params	Method	Energy Savings (%) given T'/T					
		1.05	1.1	1.2	1.3	1.4	1.5
GPT-3 2.7B	Perseus	24.5	26.0	25.9	25.2	24.6	24.0
	EnvPipe	22.9	22.6	22.0	21.4	20.9	20.4
Bloom 3B	Perseus	25.5	26.4	25.9	25.2	24.6	24.0
	EnvPipe	19.6	19.3	18.8	18.3	17.8	17.4
BERT 1.3B	Perseus	20.0	22.6	24.1	23.4	22.8	22.2
	EnvPipe	19.2	18.9	18.3	17.8	17.4	16.9
T5 3B	Perseus	27.9	27.3	26.2	25.2	24.3	23.4
	EnvPipe	18.4	18.0	17.3	16.6	16.0	15.4
Wide-ResNet 1.5B	Perseus	24.3	26.2	26.3	25.7	25.0	24.4
	EnvPipe	16.4	16.2	15.8	15.4	15.0	14.6

(b) Eight stage pipeline parallelism on A40 GPUs

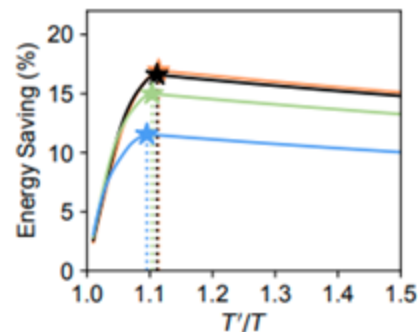
Table 4. [Experiment] Energy savings given varying straggler slowdown (T'/T). Perseus can reduce extrinsic bloat while keeping intrinsic bloat low, whereas EnvPipe cannot.

Intrinsic + Extrinsic Energy Bloat Reduction

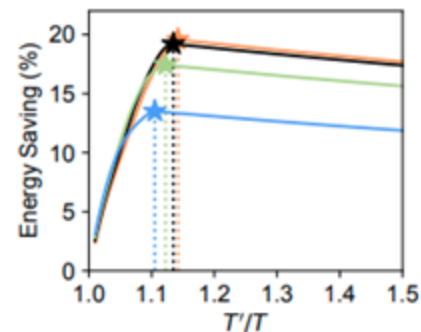


(a) GPT-3 175B on A100

(b) Bloom 176B on A100



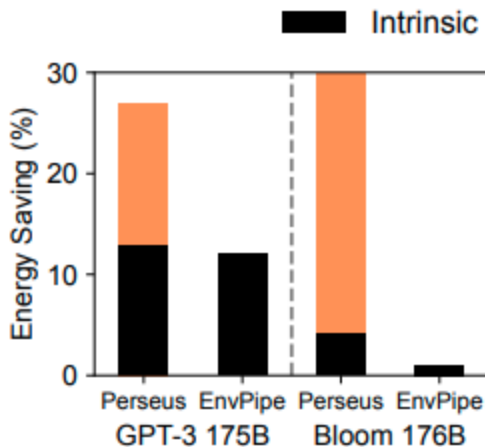
(c) GPT-3 175B on A40



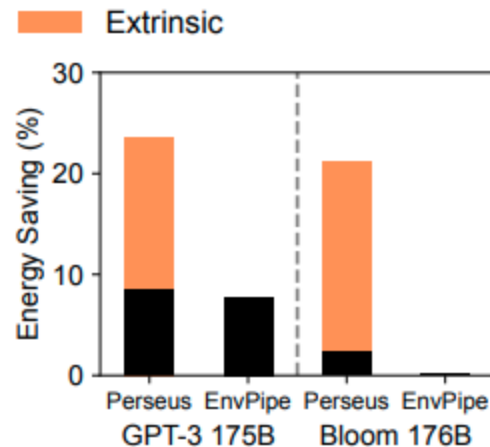
(d) Bloom 176B on A40

Scalability

- Large scale emulations
- Tests GPT-3 175B and Bloom 176B.
- Perseus saves up to 30% energy vs. EnvPipe.



(a) A100



(b) A40

Profiling overhead

Profiling

- Client-side profiler profiles forward and backward computations in each stage.
- Initial profiling phase adds 13 min overhead on A100 workloads.
- Negligible impact for large-scale model training.

Algorithm Runtime

- Average runtime across A100 workloads: 6.5 min.
- Longest runtime: Bloom 3B (15.7 min).
- GPT-3 175B (8,192 A100 GPUs):
 - 87 sec (optimizes only one data parallel pipeline).
 - Larger DAGs may increase runtime, but justified by scaling of training time.
- Optimal energy schedule lookup is instantaneous given stragglers iteration time T' .

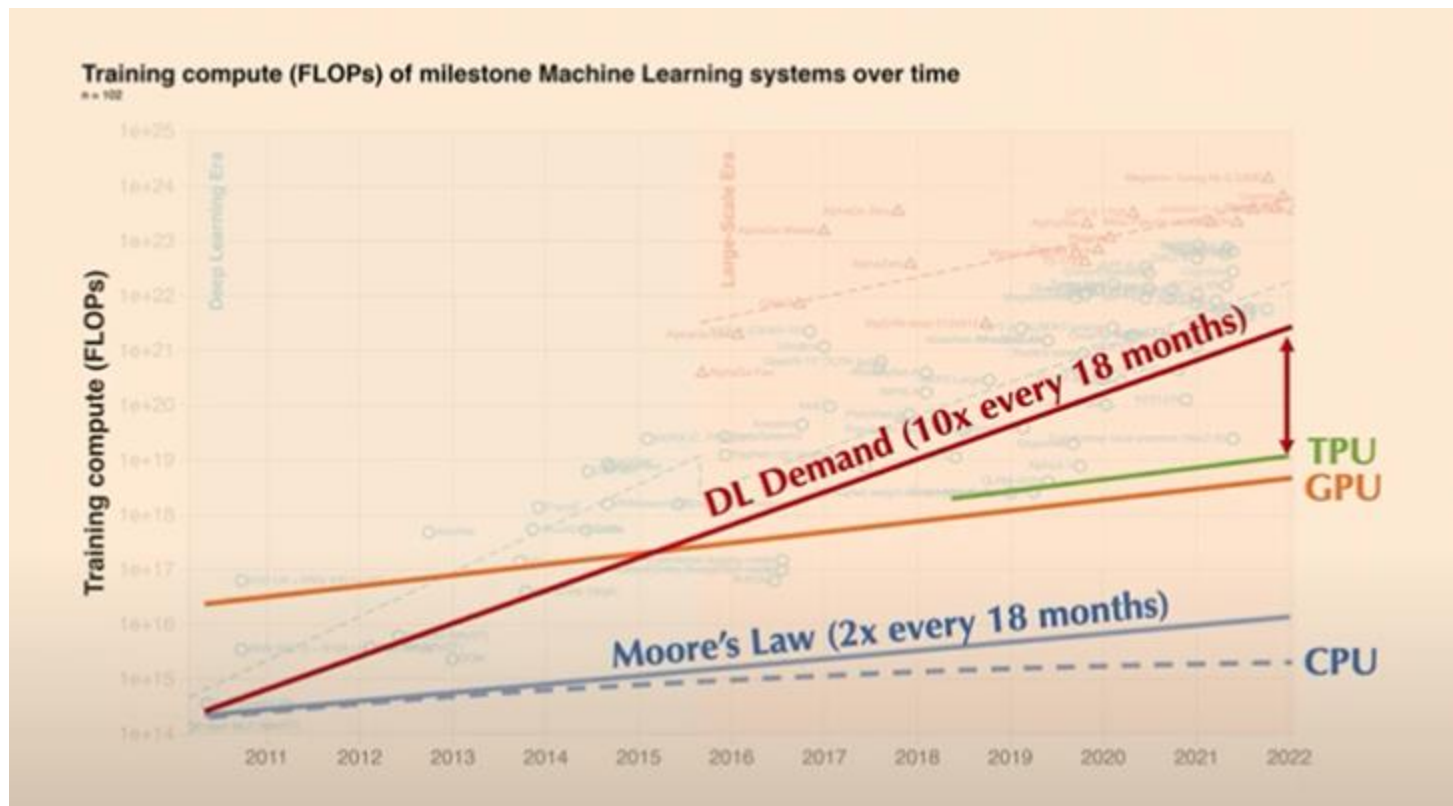
Power-aware Deep Learning Model Serving with μ -Serve

Power-aware Deep Learning Model Serving with μ -Serve

- A need to improve power efficiency:
 - Current models are focused on performance while paper focuses on improving power efficiency in regards to model serving
- Google attributes 60% of ML energy for inference
- AWS attributes inferences to 90% of ML cloud workload
- At Meta, power capacity breakdown is 1:2:7 for AI infrastructures used for Experimentation, Training, and Inference

- **purpose:** reducing energy consumption in model-serving clusters

Energy Efficiency is key for further scaling



Challenges

- 1) Difficult to determine optimal GPU frequency for both power efficiency and inference throughput
- 2) Serving non-deterministic generative models makes it hard to determine the impact of GPU frequency down scaling
- 3) GPUs and CPUs don't support fine grained frequency scaling

Key Features

- Power-aware fine-grained model provisioning
- Speculative request Serving
- SLO-preserving GPU frequency scaling

Background

- Model serving workloads are structured around a simple-response paradigm
 - Useful features are extracted through a forward pass
- A model can be trained once for an extensive sequence of inferences
- Model Parallelism and Partitioning
 - To satisfy user demand, there is a “stringent SLOs on latency”
 - 99% of requests for a model must be finished within 500ms
 - Parallelism required to store models and meet latency conditions
- Intra-operator and inter-operator parallelism in regards to energy efficiency
 - Inter-operator doesn't decrease computation latency

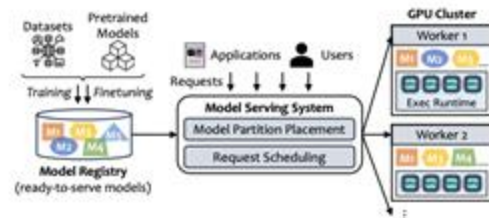


Figure 1: Model training and model serving.

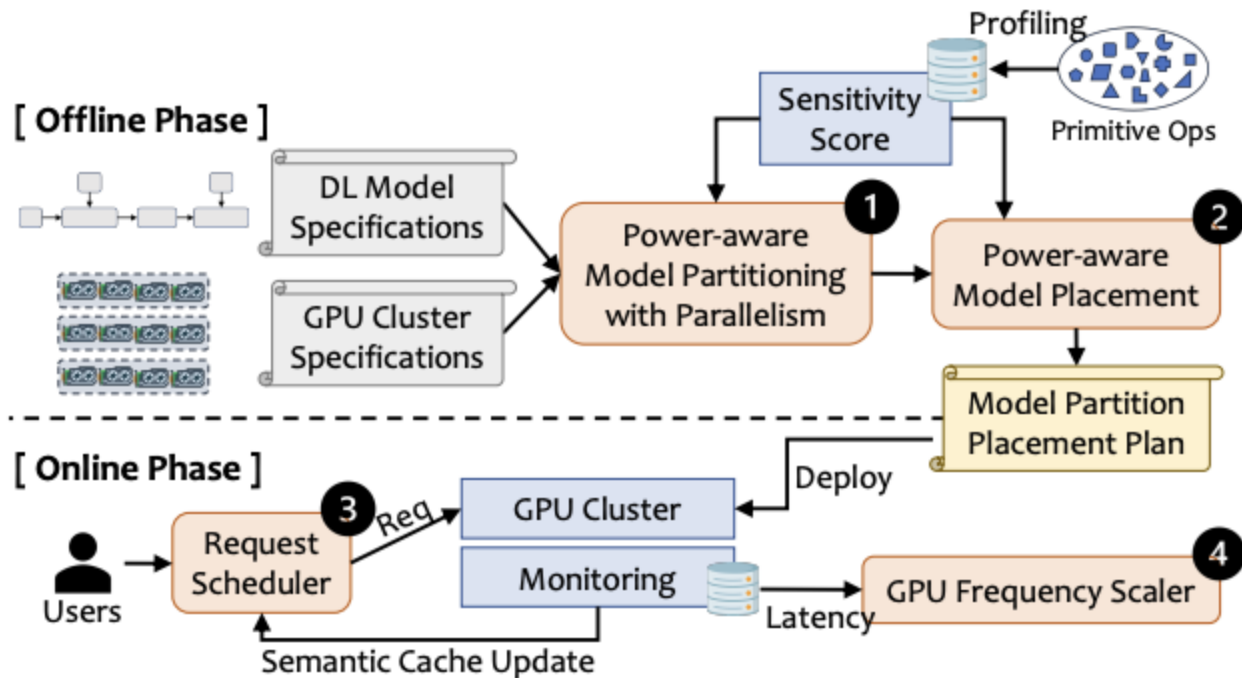


Figure 4: μ -Serve architecture overview and workflow.

μ-Serve Components

Offline:

1. Operator Sensitivity Profiling: measure how each operator's latency changes with GPU frequency.
2. Power-Aware Partitioning: cluster operators by similar sensitivity to avoid mixing highly sensitive and less sensitive ones.
3. Power-Aware Placement: distribute partitions across GPUs to maximize power-saving opportunities.

Online:

1. Speculative Scheduler for generative models (shortest-job-first with proxy predictor).
2. Dynamic GPU Frequency Scaling (multiplicative-increase-additive-decrease) ensures SLO adherence

Opportunities for power saving

Frequency vs. Latency:

- Lowering frequency significantly reduces GPU power draw but only modestly increases inference latency at certain ranges.
- Observed “valley” or plateau in latency–frequency curves for many models.

Key Leverage Point:

- Many operators or entire model partitions can sustain lower frequencies without busting SLOs, if scheduled carefully.

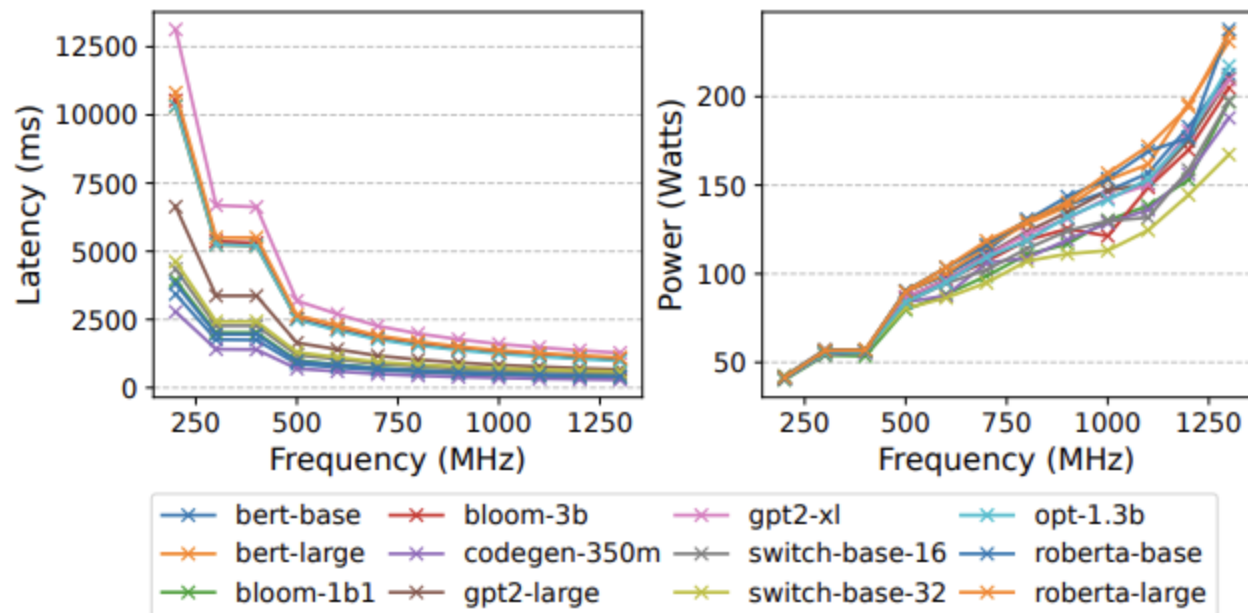


Figure 2: Model-serving latency and power consumption characterization at varying GPU core frequency levels.

Autoregressive patterns

- Non-Determinism:
 - For large language models (LLMs), each output token depends on the previously generated tokens.
 - Output token lengths vary widely per request.
- Implication:
 - Some requests may finish quickly (short responses), others run many iterations (long responses).
 - FCFS scheduling leads to unpredictably long queue times for shorter jobs.

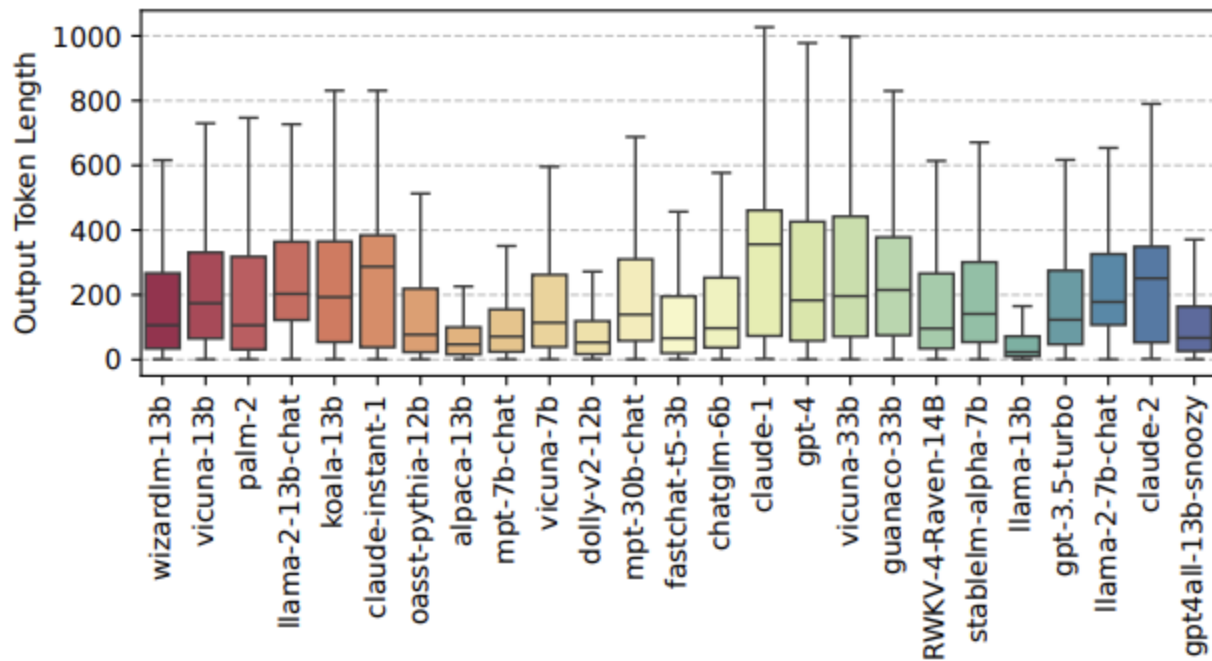


Figure 3: Output token length distributions of various large language models collected in the LMSYS-Chat-1M dataset.

Userve design and implementation

Power-Aware Fine-Grained Provisioning

- Clusters operators by sensitivity and places them to avoid “mixing” very sensitive with insensitive ones on the same GPU.

Speculative Scheduling

- Uses a lightweight BERT-based “proxy” to estimate (classify) generative output lengths.
- Implements a “shortest-job-first” approach to reduce queue delays.

SLO-Preserving Frequency Scaling

- Monitors real-time latencies and adjusts GPU frequency up or down using a feedback (MIAD) loop.

Power Aware Parallelism

Power-Aware Fine-Grained Provisioning

- Clusters operators by sensitivity and places them to avoid “mixing” very sensitive with insensitive ones on the same GPU.

Speculative Scheduling

- Uses a lightweight BERT-based “proxy” to estimate (classify) generative output lengths.
- Implements a “shortest-job-first” approach to reduce queue delays.

SLO-Preserving Frequency Scaling

- Monitors real-time latencies and adjusts GPU frequency up or down using a feedback (MIAD) loop.

Offline Profiling of Operator Sensitivity Scores

- Primitive operators in XLA's HLO format (IR)
- $\Delta\text{Latency}/\Delta\text{Frequency}$
 - $\Delta\text{Latency}$ is the increase in execution time (normalized) of the operator when decreasing the GPU core frequency from the default setting to half of the default frequency
 - $\Delta\text{Frequency}$ is the frequency decrease
- ~100 HLA primitive operators

Model Parallelism Plan Generation

- Extends AlpaServe
- Cluster the operators with similar sensitivity scores
 - Prevents operators with similar sensitivities from being split

Algorithm 1 Operator Clustering

Require: Model specification M , Operator sensitivity score database D (a map of <operator, score> pairs)

```
1: procedure OPERATORCLUSTERING( $M, D$ )
2:    $clusters \leftarrow \emptyset$ 
3:    $currCluster \leftarrow \emptyset$ 
4:    $currCluster.add(M.head)$ 
5:    $clusterScore \leftarrow avg(currCluster)$ 
6:   for each  $op$  in  $M.operators$  do
7:     if  $abs(D.get(op) - clusterScore) < THRES$  then
8:        $currCluster.append(op)$ 
9:        $clusterScore \leftarrow avg(currCluster)$ 
10:    else
11:       $clusters.append(currCluster)$ 
12:       $currCluster \leftarrow newList([op])$ 
13:    end if
14:  end for
15:  Return  $clusters$ 
16: end procedure
```

Power-aware Model Placement

- Plan Generation Step Output → map to GPU to maximize SLO attainment
 - + Optimize power-savings!
- Bin-packing problem is NP-hard (combinatorial explosion)
 - Goal: Finding the optimal $\langle \text{model partition, device group, device} \rangle$
- Mapping Models to Device Groups
 - Greedy algorithm from AlpaServe for best $\langle \text{model, device group} \rangle$
- Mapping Model Partitions to Devices
 - Insight: Largest GPU frequency reduction is bottlenecked by the most sensitive model partition on that device (hence clustering of operators by sensitivity)

Power-aware Model Placement

- What about communication overhead for spreading partitions over too many devices?
 - Empirical upper limit CT—maximum number of devices to partition onto without SLO violations

Algorithm 2 Model Partition Placement

Require: Device group DP (list of GPUs), Model partition plan MP (multi-model), D (same in Alg. 1), CT (maximum number of devices to place to))

```
1: procedure MODELPLACEMENT( $DP, MP, D$ )
2:    $bins \leftarrow \emptyset$   $\triangleright$  a map of device to a list of partitions
3:    $modelBinsCount \leftarrow \emptyset$ 
4:   for each  $model$  in  $MP$  do
5:     // Sort partitions by sensitivity scores
6:     for each  $p$  in  $model.getSortedPartitions()$  do
7:        $bestDevice = \text{None}$ 
8:        $bestScore = INFINITY$ 
9:        $selectedBins = \text{getAvailableBins}(bins, p)$ 
10:      for each  $d, partitions$  in  $selectedBins$  do
11:        if  $modelBinsCount[model] < CT$  then
12:          // Get similarity in sensitivity scores
13:           $score \leftarrow \text{getSimilarity}(p, partitions)$ 
14:          if  $bestScore < score$  then
15:             $bestScore = score$ 
16:             $bestDevice = d$ 
17:          end if
18:        end if
19:      end for
20:       $bins[bestDevice].append(p)$ 
21:       $modelBinsCount[model] += 1$ 
22:    end for
23:  end for
24:  Return  $bins$ 
25: end procedure
```

Scheduler (Online)

- FCFS for traditional models
- SJF for autoregressive models
 - Need a good estimate!
 - Use trained output token length predictor
- Semantic cache (GPTCache) for hot input queries
 - Embedding $\rightarrow$ output token length
- Use final hidden state (CLS) instead of input sequence for performance

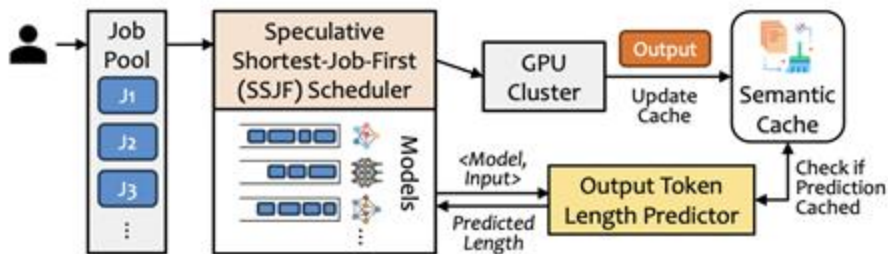


Figure 5: Workflow of the speculative scheduling in μ -Serve.

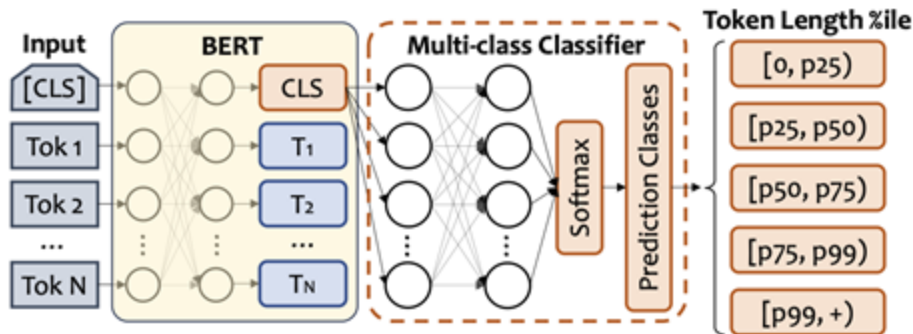


Figure 6: Output token length predictor in μ -Serve.

GPU Frequency Scaling

- Multiplicative-increase-additive-decrease (MIAD) based off of feedback signal (target SLO latency - actual)
 - Inspired by AIMD in TCP congestion control
- 500ms sampling
 - Up-scaling action doubles current frequency
 - Typically need to aggressively scale up to **mitigate SLO violations**
 - Down-scaling action decreases frequency by *STEP*

Evaluation

- Does μ -Serve achieve power-saving (and how much) while preserving SLO attainment?
- How much does the proxy-model-based scheduler in μ -Serve improve latency and throughput?
- How robust is μ -Serve's power-aware model partitioning and placement under model variations?

Model	# of Params	Size	Latency	AR?
ResNet-50	25M	0.2 GB	51 ms	No
BERT-base	110 M	0.5 GB	123 ms	No
BERT-large	340 M	1.4 GB	365 ms	No
RoBERTa-base	125 M	0.5 GB	135 ms	No
RoBERTa-large	355 M	1.4 GB	382 ms	No
OPT-1.3b	1.3 B	5.0 GB	1243 ms	Yes
OPT-2.7b	2.7 B	10.4 GB	2351 ms	Yes
GPT2-large	774 M	3.3 GB	832 ms	Yes
GPT2-xl	1.5 B	6.4 GB	1602 ms	Yes
CodeGen-350m	350 M	1.3 GB	357 ms	Yes
CodeGen-2b	2.0 B	8.0 GB	2507 ms	Yes
Bloom-1b1	1.1 B	4.0 GB	523 ms	Yes
Bloom-3b	3.0 B	11.0 GB	1293 ms	Yes
Switch-base-16	920 M	2.4 GB	348 ms	Yes
Switch-base-32	1.8 B	4.8 GB	402 ms	Yes

Does it work?

AlpaServe-Sched: only μ -Serve's scheduler

AlpaServe-Scaling: only μ -Serve's frequency scaling

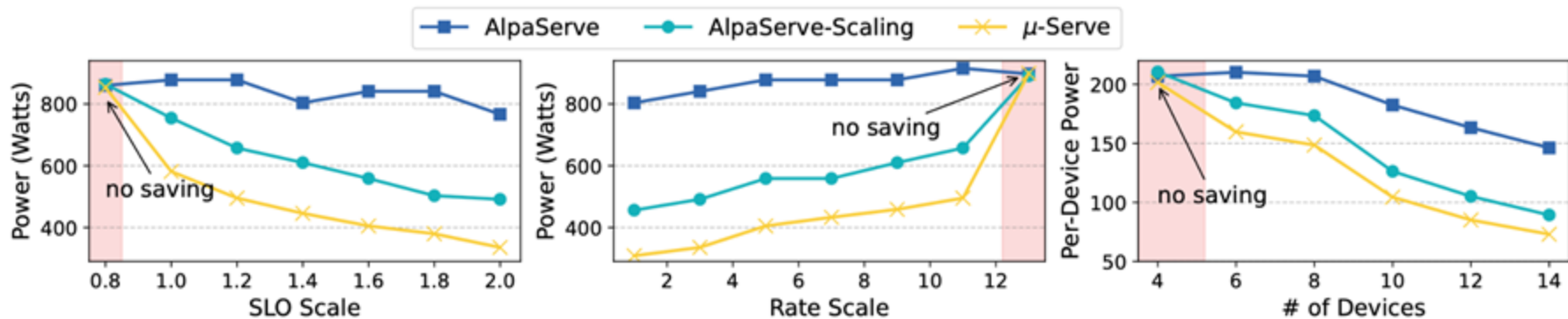


Figure 7: Power saving evaluation.

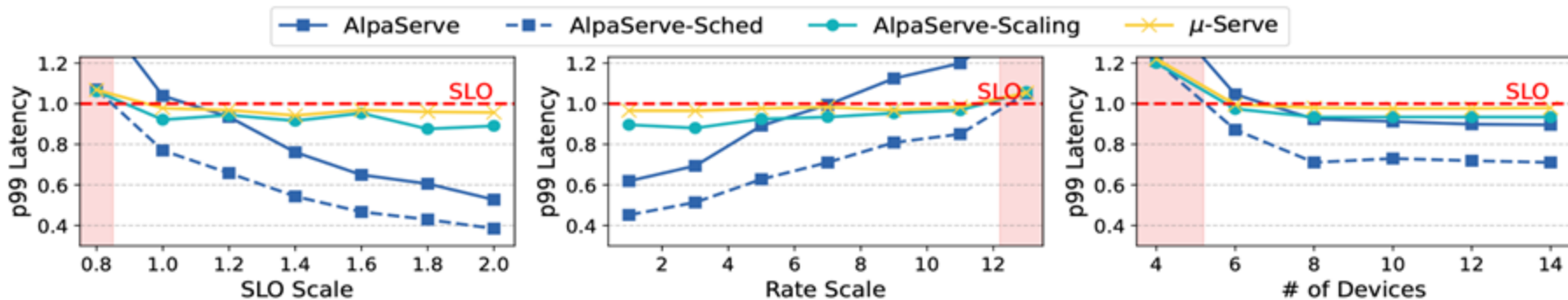


Figure 8: SLO preservation evaluation.

Evaluation

- Does μ -Serve achieve power-saving (and how much) while preserving SLO attainment?
- **How much does the proxy-model-based scheduler in μ -Serve improve latency and throughput?**
- How robust is μ -Serve's power-aware model partitioning and placement under model variations?

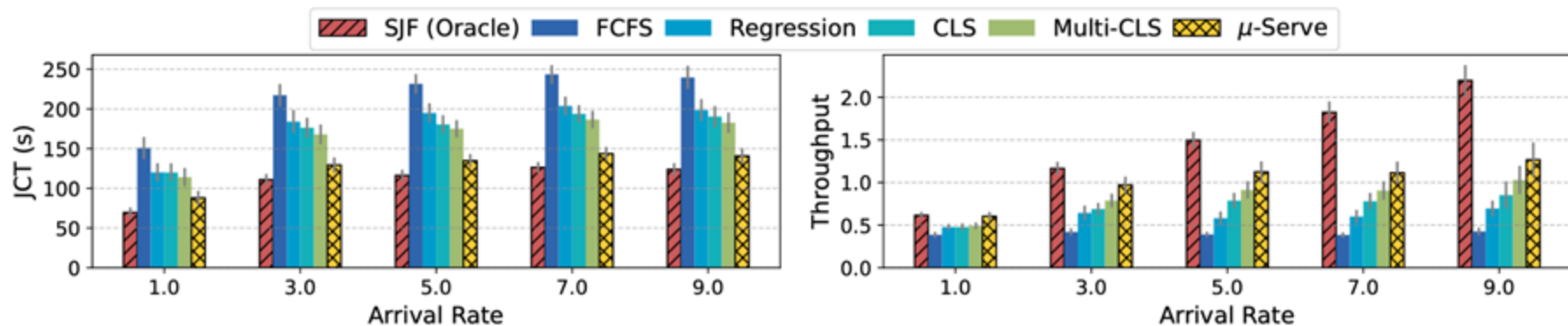


Figure 9: Scheduler evaluation across varying request arrival rates.

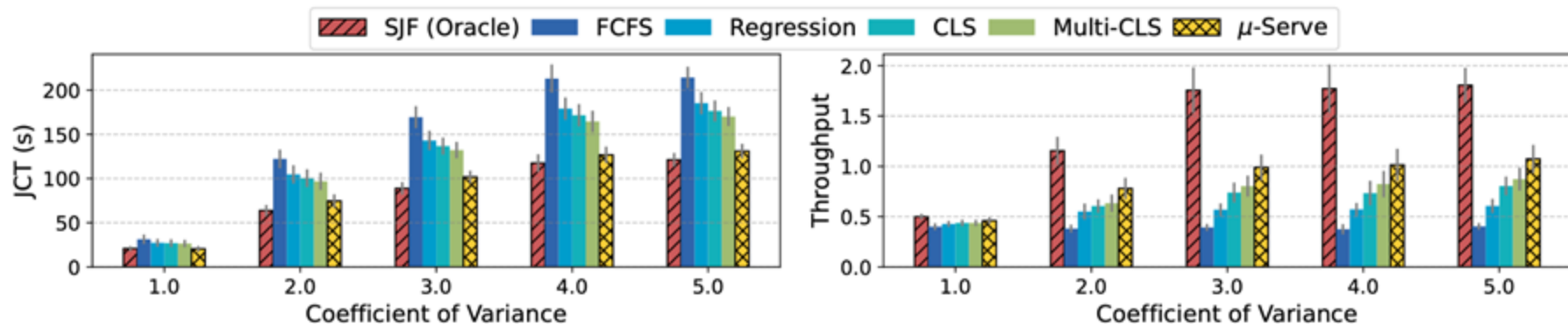


Figure 10: Scheduler evaluation across varying request burstiness.

Predictor Performance

Table 2: Output token length predictor evaluation results. Columns are five predictor types: ordinal classification with MSE/L1 loss, classification, and regression with MSE/L1 loss.

Metrics	Ord. CLS (MSE)	Ord. CLS (L1)	CLS	REG (MSE)	REG (L1)
Accuracy $\uparrow$	0.4290	0.4272	0.4599	0.4014	0.4383
F1 Score $\uparrow$	0.3563	0.3332	0.4021	0.3318	0.3477
L1 Error $\downarrow$	0.7587	0.7480	N/A	0.6077	0.5557
MSE $\downarrow$	0.9859	1.0857	N/A	0.8077	0.8996

- “Better prediction accuracy does not necessarily translate to better scheduling performance. With negligible overhead, μ -Serve reduces the average JCT by up to 41.3% and increases the throughput by up to 2.5 $\times$ at varying workloads.”

Evaluation

- Does μ -Serve achieve power-saving (and how much) while preserving SLO attainment?
- How much does the proxy-model-based scheduler in μ -Serve improve latency and throughput?
- **How robust is μ -Serve's power-aware model partitioning and placement under model variations?**

Power Saving Opportunity vs. Insensitivity Ratio

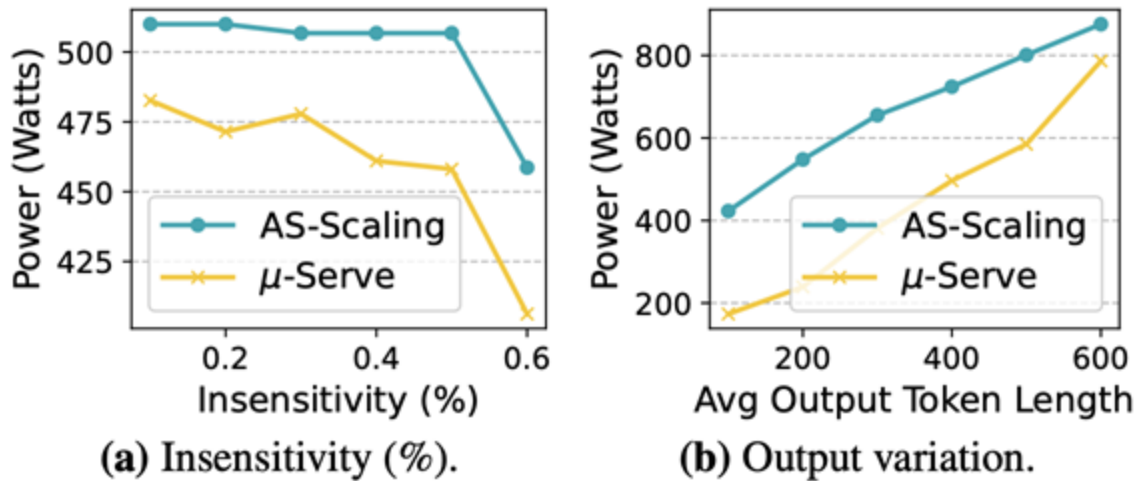


Figure 14: Robustness analysis.

- μ -Serve: relatively bigger gains when there are more insensitive operators!

Robustness to Model Execution Non-determinism

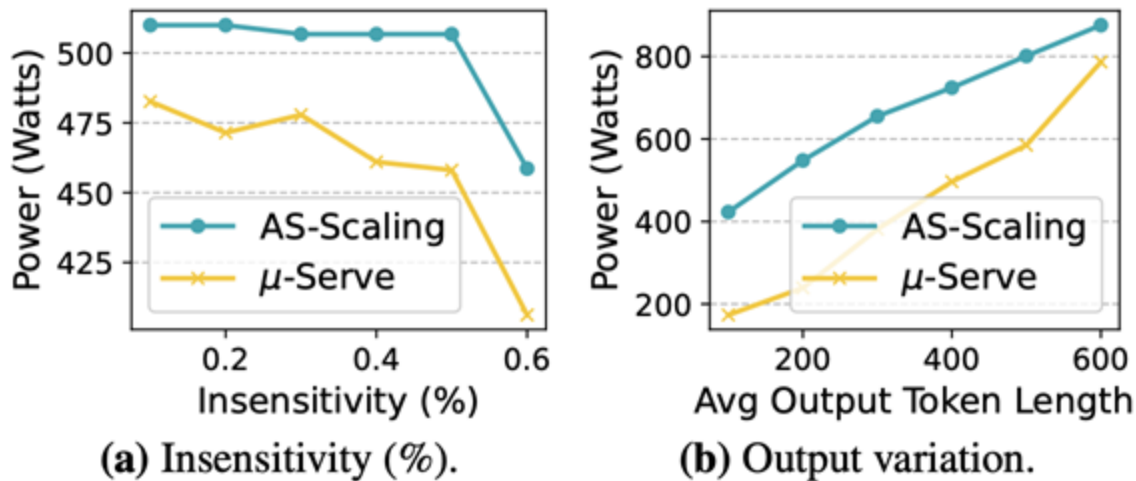


Figure 14: Robustness analysis.

- As output length increases, there are less opportunities for power-saving due to more intensive model execution

Further Discussion

- Request batching and autoscaling
 - (adding/removing model instances) are two essential orchestration tasks
 - Complementary to μ -Serve
- Heterogeneous Accelerators
 - Not just a single type of GPU!
 - Not covered
- Other Power Management Features
 - Only considered SM frequency scaling (fast actuation)
 - What about power capping, memory frequency scaling, MIG sharing?
- Scalability

Thank you!